

Hit me with your best shot - Portable node stress testing with Torch Hammer

Isa Wazirzada

April 25, 2026



Before we begin...

No Silver Bullet

Essence and Accidents of Software Engineering

Frederick P. Brooks, Jr.

University of North Carolina at Chapel Hill

Fashioning complex conceptual constructs is the *essence*; accidental tasks arise in representing the constructs in language. Past progress has so reduced the accidental tasks that future progress now depends upon addressing the *essence*.

Of all the monsters that fill the nightmares of our folklore, none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, one seeks bullets of silver that can magically lay them to rest.

The familiar software project, at least as seen by the nontechnical manager, has something of this character; it is usually innocent and straightforward, but is capable of becoming a monster of missed schedules, blown budgets, and flawed products. So we hear desperate cries for a silver bullet—something to make software costs drop as rapidly as computer hardware costs do.

But, as we look to the horizon of a decade hence, we see no silver bullet. There is no single development, in either technology or in management technique, that by itself promises even one order-of-magnitude improvement in productivity, in reliability, in simplicity. In this article, I shall try to show why, by examining both the nature of the software problem and the properties of the bullets proposed.

Skepticism is not pessimism, however. Although we see no startling break-

throughs—and indeed, I believe such to be inconsistent with the nature of software—many encouraging innovations are under way. A disciplined, consistent effort to develop, propagate, and exploit these innovations should indeed yield an order-of-magnitude improvement. There is no royal road, but there is a road.

The first step toward the management of disease was replacement of demon theories and humours theories by the germ theory. That very step, the beginning of hope, in itself dashed all hopes of magical solutions. It told workers that progress would be made stepwise, at great effort, and that a persistent, unremitting care would have to be paid to a discipline of cleanliness. So it is with software engineering today.

Does it have to be hard?—Essential difficulties

Not only are there no silver bullets now in view, the very nature of software makes it unlikely that there will be any—no inventions that will do for software productivity, reliability, and simplicity what electronics, transistors, and large-scale integration did for computer hardware.

This article was first published in *Information Processing '86*, ISBN No. 0-444-70077-3, H.-J. Kugler, ed., Elsevier Science Publishers B.V. (North-Holland) © IFIP 1986.

Background

Motivation

Context: Tracking component health during bringup and over a system's service life is not easy

Challenging to pivot between different hardware architectures

- Verify component performance (GPU, APU, CPU)
- Identify thermal issues via high-frequency telemetry
- Tools often have tight coupling to vendor specific programming environments

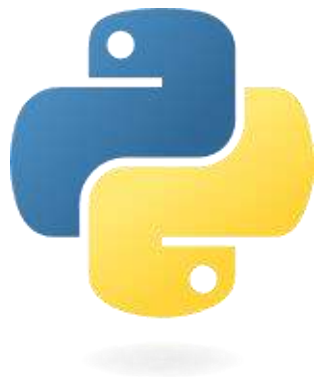
Inspiration

- Diverse universe of workloads, datatypes, architectures
- Recurring need for a tool that can characterize the performance of hardware, but also capture telemetry:
 - Temperature
 - CU/SM utilization, clock frequencies, memory bandwidth utilization, etc.
 - Run time software environment
 - Present the telemetry and collected metadata in a consistent format that is agnostic of the hardware being stressed

Prolog to Torch Hammer

- One precision, one hardware architecture, one programming environment, 20,000 reasons to collect rich metadata
- 20,000 GPUs Under the DGEMM Sea, Thursday April 30, 13:45-14:07, large scale performance systems track





License Apache 2.0 tests 521 passed coverage 76% python 3.8+ PyTorch 1.10+



Forged with PyTorch
GPU/CPU/APU Micro-Benchmark Suite

<https://github.com/HPE/torch-hammer>

Torch Hammer Highlights

Cross-platform: CUDA, ROCm, Metal (MPS), CPU – automatically selected.

Nine micro benchmarks

- Batched GEMM (matrix multiply)
- 2-D Convolution
- 3-D FFT
- Einsum (attention-style tensor contraction)
- Random memory traffic
- Laplacian heat-equation solver
- 1-D time-dependent Schrödinger equation
- Atomic contention (L2 cache stress via scatter_add)
- Sparse matrix multiplication (SpMM)

Precision choices: float16, bfloat16, float32/TF32 mode, float64, complex64, complex128.

Live telemetry

— NVIDIA/AMD: power, clock, memory controller activity, memory utilization, GPU temperature, SM/CU utilization, throttle state

Verbose logging: every iteration, every telemetry field, and a variety of outputs (CSV, JSON, Standard Out, Journal)

CI: 521 pytest + 96 ReFrame checks



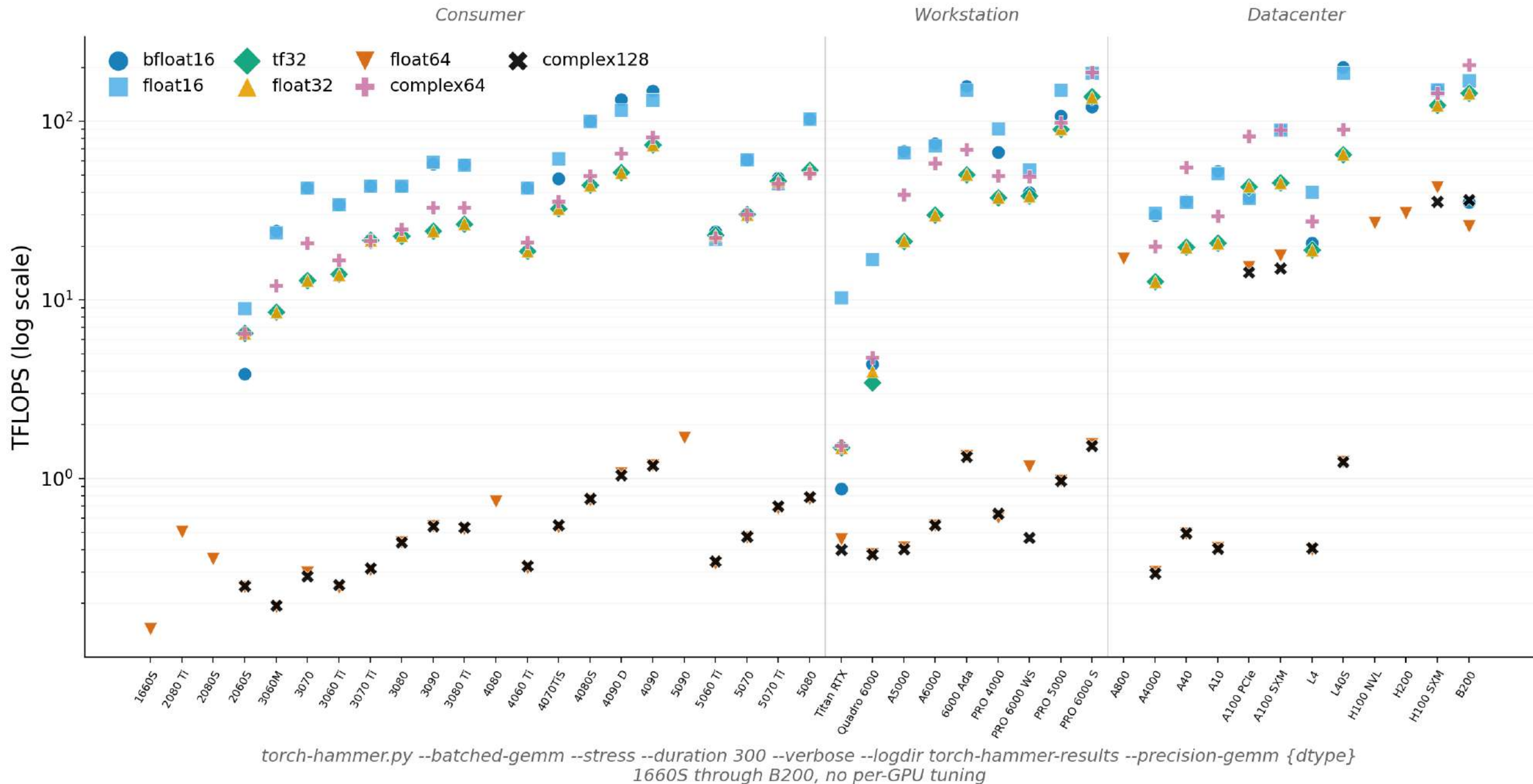
Platforms tested

Category	NVIDIA	AMD
Data Center	GB300, GH200, B200, H200, H100, A100 (PCIe & SXM), L40S, L4, A40, A10	MI300A, MI250X
Workstation	RTX 6000 Ada, A6000, A5000, Quadro RTX 6000	
Consumer	GeForce GTX 1660S through RTX 5080 (26 GPUs)	
CPU	Grace	EPYC (Rome, Milan, Trento)

Other platforms include: Apple M3 Pro, ARM Cortex-A76



One Tool, 43 GPUs, 7 Precisions



Quick start – Nvidia Example

```
pip install torch --index-url https://download.pytorch.org/whl/cu130

# Optional libraries
pip install nvidia-ml-py setproctitle pyyaml
```

Launching Torch Hammer against a node for 90 seconds

```
./torch-hammer.py --stress --batched-gemm --all-gpus --compact --duration 90
hostname,gpu,gpu_model,serial,benchmark,dtype,iterations,runtime_s,min,mean,max,unit,power_avg_w,temp_max_c
node1,0,NVIDIA GB300,1641825003329,Batched GEMM,float32,29,90.117,69179.0882,69185.2975,69192.6416,GFLOP/s,1039.9,49
node1,1,NVIDIA GB300,1641025004788,Batched GEMM,float32,29,90.088,69200.9415,69208.1724,69212.9451,GFLOP/s,1041.6,50
node1,2,NVIDIA GB300,1641225000544,Batched GEMM,float32,29,90.041,69242.5367,69244.8847,69250.0942,GFLOP/s,1050.1,50
node1,3,NVIDIA GB300,1641725002897,Batched GEMM,float32,29,90.074,69210.0920,69214.8037,69237.5359,GFLOP/s,1031.9,49
```

Configuration options – Command-Line Reference

Core Execution

- `--config <yaml>` — Benchmark configuration
- `--all-gpus / --gpu-list 0,2,3`
- `--duration <sec>` — Time-based runs
- `--warmup <n>` — Pre-timing iterations
- `--stress-test` — Auto-scale problem sizes
- `--shuffle` — Randomize execution order

Output & Integration

- `--compact` — One-row-per-benchmark CSV (stdout)
- `--json-output <file>` — Full telemetry + metadata
- `--summary-csv <file>` — Fleet-level summary
- `--syslog / --syslog-dmesg`
- `--log-file / --log-dir / --no-log`

Multi-GPU & NUMA Awareness

- `--cpu-affinity`
- `--cpu-gpu-map 0:0-15,1:16-31`
- `--startup-delay-per-gpu <sec>`
- `--parent-cpu <core>`
- `--cpu-list <cores>` — CPU-only mode

Telemetry & Validation

- `--telemetry-interval-ms` (*default: 100*)
- `--skip-telemetry-first-n`
- `--temp-warn-C / --temp-critical-C`
- `--power-warn-pct`
- `--outlier-threshold-pct`
- `--efficiency-warn-pct`
- `--baseline-file`
- `--no-validation`

Configuration options – YAML File

```
# Global settings
verbose: true
log_file: "stress-test.log"
device_index: 0
warmup: 20
repeats: 3
repeat_delay: 10
# Output modes (uncomment to enable)
# compact: true           # Machine-readable CSV to stdout
# syslog: true            # Structured key=value entries to syslog
# syslog_dmesg: true      # Also write to /dev/kmsg (requires syslog: true)
```

`./torch-hammer.py --config config.yaml`

```
# Benchmark suite – can specify same test multiple times with different params
benchmarks:
- name: batched_gemm
  precision: float32
  batch_count: 128
  m: 4096
  n: 4096
  k: 4096
  inner_loop: 100

- name: batched_gemm
  precision: float64
  batch_count: 64
  m: 8192
  n: 8192
  k: 8192
  TF32_mode: false

- name: convolution
  precision: bfloat16
  batch_count: 256
  in_channels: 64
  out_channels: 128

- name: fft
  precision: float32
  nx: 256
  ny: 256
  nz: 256

- name: heat_equation
  precision: float64
  grid_size: 32768
  time_steps: 100
```

Baselines – Setting performance thresholds for the fleet

```
# The performance figures below are examples, please check vendor  
# datasheets for specific figures
```

```
"NVIDIA GH200 120GB":
```

```
  fp32_tflops: 51.0      # FP32 peak TFLOPS  
  fp64_tflops: 25.5      # FP64 peak TFLOPS  
  tf32_tflops: 756.0     # TF32 peak TFLOPS (Tensor Core)  
  memory_bandwidth_gbps: 4800.0 # HBM3e bandwidth in GB/s  
  tdp_watts: 900         # Thermal Design Power in Watts
```

```
"AMD Instinct MI300X":
```

```
  fp32_tflops: 163.4  
  fp64_tflops: 81.7  
  memory_bandwidth_gbps: 5300.0  
  tdp_watts: 750
```

```
"NVIDIA A100-SXM4-80GB":
```

```
  fp32_tflops: 19.5  
  fp64_tflops: 9.7  
  tf32_tflops: 156.0  
  memory_bandwidth_gbps: 2039.0  
  tdp_watts: 400
```

```
"AMD Instinct MI250X":
```

```
  fp32_tflops: 47.9  
  fp64_tflops: 47.9  
  memory_bandwidth_gbps: 3276.8  
  tdp_watts: 500
```


Single Node Interactive Summary

2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	MULTI-GPU SUMMARY
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	BENCHMARK RESULTS - ALL TESTS
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	Test: Batched GEMM
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	Parameters: dtype=complex128, batch=64, MxNxK=3156x3156x3156
2025-12-22T12:52:05 INFO	Performance (GFLOP/s): min / mean / max
2025-12-22T12:52:05 INFO	GPU 0: 36047.18 / 36268.09 / 36775.94
2025-12-22T12:52:05 INFO	GPU 1: 35732.03 / 36141.57 / 36589.48
2025-12-22T12:52:05 INFO	GPU 2: 35593.68 / 35878.13 / 36414.26 [SLOWEST]
2025-12-22T12:52:05 INFO	GPU 3: 36067.09 / 36456.22 / 36982.59 [FASTEST]
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	Telemetry Statistics:
2025-12-22T12:52:05 INFO	GPU 0:
2025-12-22T12:52:05 INFO	SM Util: 0% / 100% / 100%
2025-12-22T12:52:05 INFO	Mem BW Util: 0% / 76% / 85%
2025-12-22T12:52:05 INFO	Temperature: 34°C / 49.1°C / 51°C
2025-12-22T12:52:05 INFO	Power: 126W / 553W / 580W
2025-12-22T12:52:05 INFO	Memory Used: 1.1GB / 29.7GB / 29.7GB
2025-12-22T12:52:05 INFO	GPU 1:
2025-12-22T12:52:05 INFO	SM Util: 2% / 100% / 100%
2025-12-22T12:52:05 INFO	Mem BW Util: 0% / 74% / 82%
2025-12-22T12:52:05 INFO	Temperature: 36°C / 53.5°C / 56°C
2025-12-22T12:52:05 INFO	Power: 131W / 559W / 588W
2025-12-22T12:52:05 INFO	Memory Used: 1.1GB / 29.7GB / 29.7GB
2025-12-22T12:52:05 INFO	GPU 2:
2025-12-22T12:52:05 INFO	SM Util: 0% / 100% / 100%
2025-12-22T12:52:05 INFO	Mem BW Util: 0% / 74% / 83%
2025-12-22T12:52:05 INFO	Temperature: 36°C / 52.5°C / 54°C
2025-12-22T12:52:05 INFO	Power: 129W / 557W / 580W
2025-12-22T12:52:05 INFO	Memory Used: 1.1GB / 29.7GB / 29.7GB
2025-12-22T12:52:05 INFO	GPU 3:
2025-12-22T12:52:05 INFO	SM Util: 2% / 100% / 100%
2025-12-22T12:52:05 INFO	Mem BW Util: 0% / 75% / 81%
2025-12-22T12:52:05 INFO	Temperature: 34°C / 51.4°C / 53°C
2025-12-22T12:52:05 INFO	Power: 119W / 556W / 581W
2025-12-22T12:52:05 INFO	Memory Used: 1.1GB / 29.7GB / 29.7GB
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	BENCHMARK SUMMARY
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	Test: Batched GEMM (complex128)
2025-12-22T12:52:05 INFO	GPU Serial Performance Power(avg) Temp(max) Status Notes
2025-12-22T12:52:05 INFO	-----
2025-12-22T12:52:05 INFO	0 1651223001955 36268.1 GFLOP/s 553W 51C PASS
2025-12-22T12:52:05 INFO	1 1650174001501 36141.6 GFLOP/s 559W 56C PASS
2025-12-22T12:52:05 INFO	2 1651524004934 35878.1 GFLOP/s 557W 54C PASS
2025-12-22T12:52:05 INFO	3 1650324005519 36456.2 GFLOP/s 556W 53C PASS Fastest
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	Aggregate: 144744.0 GFLOP/s across 4 GPUs Variation: CV=0.7%
2025-12-22T12:52:05 INFO	=====
2025-12-22T12:52:05 INFO	JSON results exported to: torchhammer_results/20251222_125026_testing/nid001000/results_nid001000_20251222_125205.json
2025-12-22T12:52:05 INFO	=====

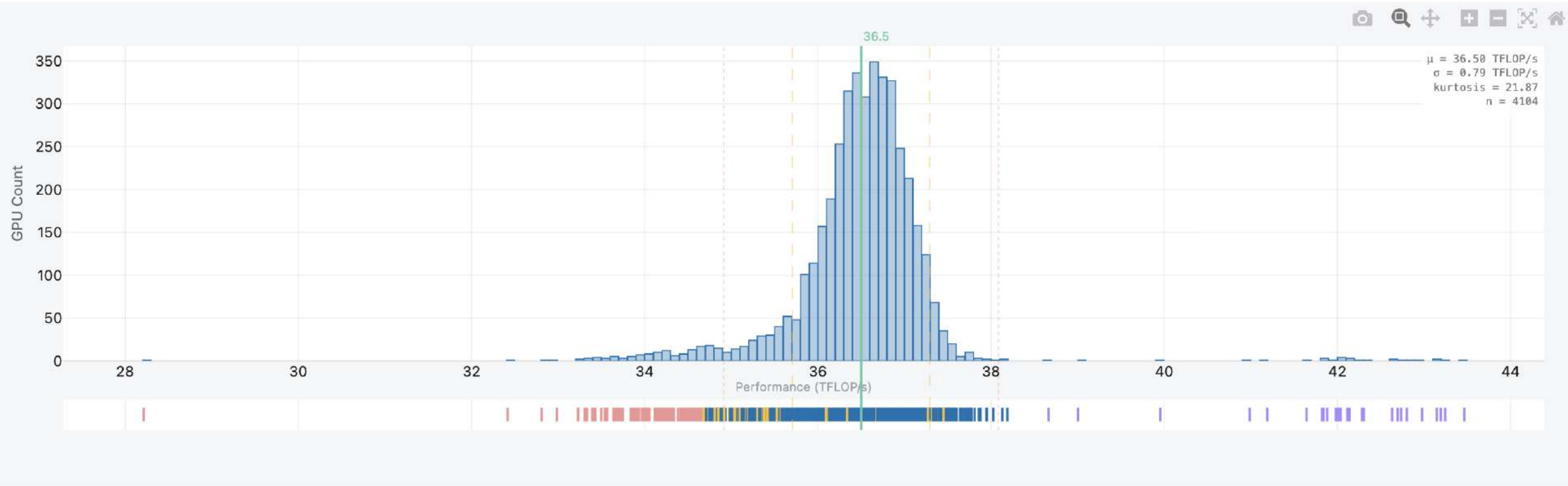
Per test summary of key telemetry

Performance summary

Interactive Torch Hammer Report: hammer-report.py

Fleet Distribution

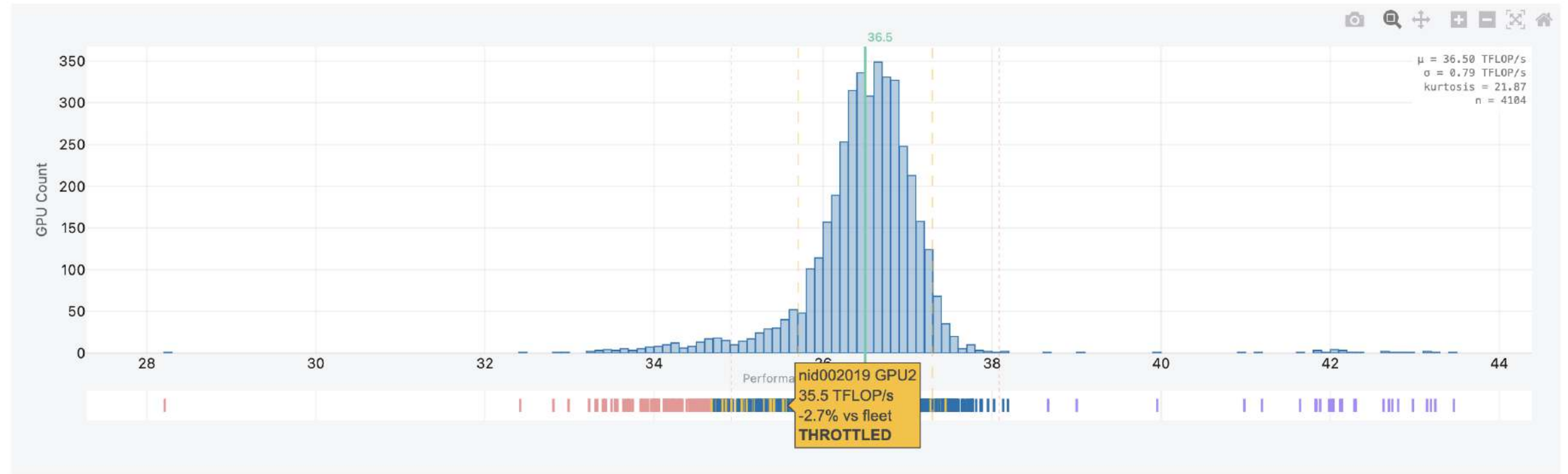
Tight bell curve = consistent fleet. Long tails or secondary peaks = subpopulations worth investigating. GPUs beyond $\pm 2\sigma$ (red) are statistical outliers.



Interactive Torch Hammer Report: hammer-report.py

Fleet Distribution

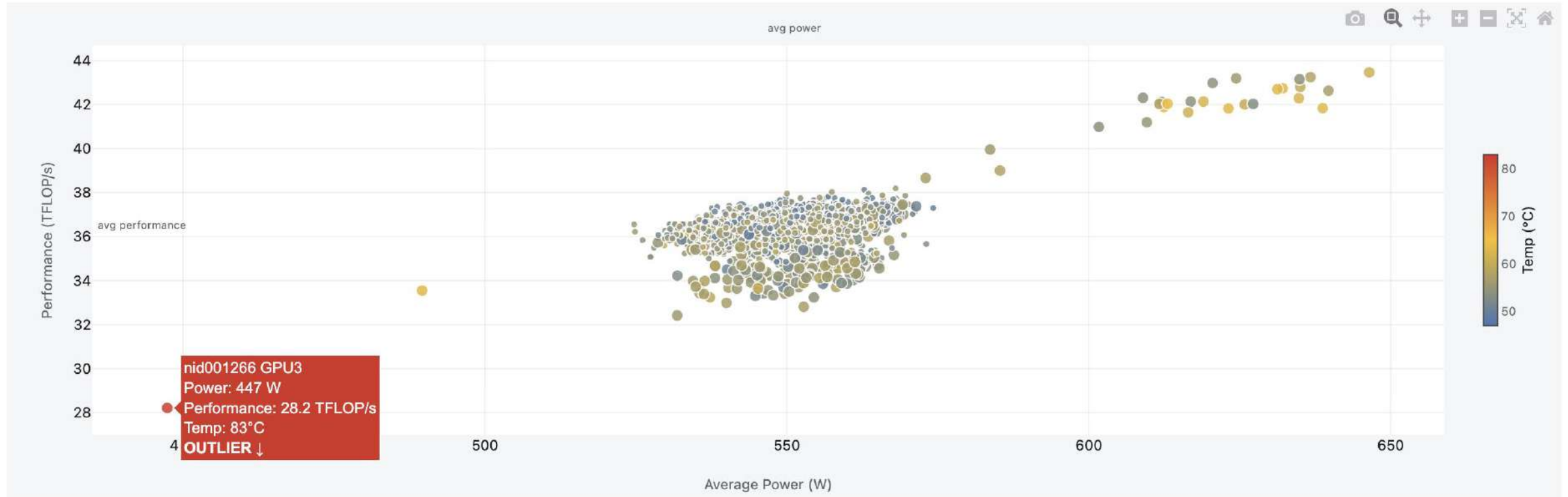
Tight bell curve = consistent fleet. Long tails or secondary peaks = subpopulations worth investigating. GPUs beyond $\pm 2\sigma$ (red) are statistical outliers.



Interactive Torch Hammer Report: hammer-report.py

Power vs Performance

Reveals whether power limits performance. Color = temperature (blue cool, red hot). Larger dots = outlier or throttled GPUs. Tight cluster = healthy. Horizontal spread at fixed performance = power variation without impact. Diagonal trend = power-limited GPUs.



Interactive Torch Hammer Report: hammer-report.py

Fleet Map

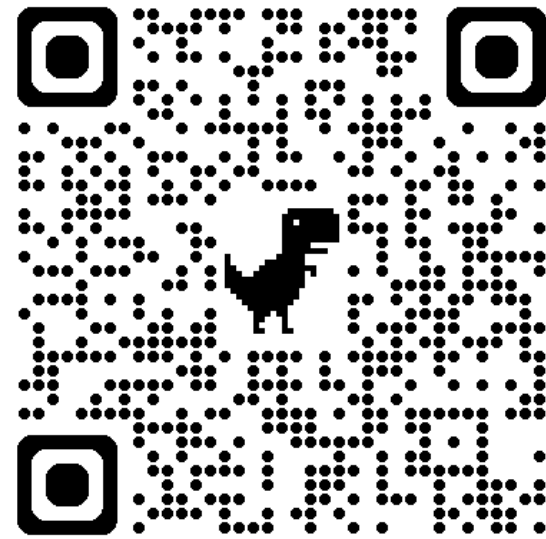
Spots topology-correlated problems. Uniform color = healthy fleet. Clusters of red/blue within a cabinet = localized issue (cooling, power, hardware fault).

low (bad)  high (good) mean: 36.5 TFLOP/s  outlier ↓  outlier ↑  throttled



Thank You

Isa Wazirzada, isa.Wazirzada@hpe.com



<https://github.com/HPE/torch-hammer>

Can you find the easter egg?